

第一章 作业

第一章

1.1 (1) 虚拟机: 由软件实现的机器

(2) 摩尔定律: 集成芯片上所集成的晶体管数目每隔 18 个月就翻一番。

(3) 透明性: 在计算机技术中, 把这种本来存在的事物或属性, 但从某种角度看似不存在, 的概念称为透明性。

(4) CPI: 每条指令执行的平均时钟周期数 (Cycles Per Instruction)

$CPI = \text{执行所需的时钟周期数} / IC$ IC 是所执行的指令条数。

(5) 松散耦合系统 (间接耦合系统): 一般是通过通信线路实现计算机之间的互连, 可共享外存设备 (磁盘, 磁带等)。机器之间的相互作用是在文件或数据块一级上进行的。

1.3

Plymm 分类法按照指令流和数据流的多倍性进行分类。

分为: (1) 指令流: 计算机执行的指令序列

(2) 数据流: 由指令流调用的数据序列

(3) 多倍性: 在系统度量的部件上, 同时处于同一执行阶段的指令或数据的最大数目。

1.6

1.6 某台主频为 400 MHz 的计算机执行标准测试程序, 程序中指令类型、执行数量和平均时钟周期数如表 1.2 所示。

表 1.2 标准测试程序中的各项指标

指令类型	指令执行数量	平均时钟周期数
整数	45 000	1
数据传送	75 000	2
浮点	8 000	4
分支	1 500	2

求该计算机的有效 CPI、MIPS 和程序执行时间。

$$\text{总指令数} = 45000 + 75000 + 8000 + 1500 = 127500 \text{ 条}$$

$$\text{总周期数} = 45000 + 75000 \times 2 + 8000 \times 4 + 1500 \times 2 = 230000$$

$$230000 / 127500 = 1.8039 \text{ CPI}$$

$$[127500 / (230000 / 400 \cdot 10^6)] / 10^6 = 21.74 \text{ MIPS}$$

$$\text{时间} = \frac{230000}{400 \times 10^6} = 5.75 \times 10^{-4} \text{ s}$$

1.7

$$S = \frac{1}{(1-f) + f/n} = \frac{1}{(1-0.4) + 0.4/10} = 1.5625 \text{ 倍}$$

第二章 作业

2.1 1) CISC (Complex Instruction Set Computer - 复杂指令集计算机):

设计策略为增强指令功能,把越来越多的功能交由硬件来实现,并且指令的数量越来越多。

2) RISC (Reduced Instruction Set Computer - 精简指令集计算机):

尽可能地简化指令集,不仅指令的条数少,而且指令的功能也比较简单。

3) 寻址方式: 一种指令集结构如何确定所要访问的数据的地址的方法。

4) 数据表示: 计算机硬件能够直接识别、指令集可以直接调用的数据类型。

5) 通用寄存器计算: CPU内部有一组可以自由使用的寄存器。

根据操作数的来源不同,又可进一步分为:

寄存器-存储器结构(RM结构): 操作数可以来自寄存器。

寄存器-寄存器结构(RR结构): 所有操作数都来自通用寄存器值,也称为load-store结构。这名称强调只有load指令和store指令能够访问存储器。

2.4 对指令集的基本要求: 完整性、规整性、高效率、兼容性。

2.7 设计RISC和器遵循的原则:

1. 指令条数少而简单,只选取使用频率很高的指令,在此基础上补充一些最有用的指令。
2. 采用简单而又统一的指令格式,并减少寻址方式;指令长度都为32位或64位。
3. 指令的执行在单周期内完成。(采用流水技术)
4. 只有load和store指令才能访问存储器,其他指令的操作都是在寄存器之间进行。

5. 大多数指令都用硬连线逻辑来实现。

6. 强调用编译器的作用,为高级语言程序生成优化的代码。

7. 充分利用流水技术来提高性能。

第二章 实验一: MIPS 指令系统和体系结构

一、实验目的

1. 了解和熟悉指令级模拟器

2. 熟悉掌握MIPSsim模拟器的操作和使用方法。

3. 熟悉MIPS指令系统及其特点,加深对MIPS指令操作语义的理解。

4. 熟悉MIPS体系结构。

二、实验步骤与记录

A.1.3 实验内容和步骤

首先要阅读 MIPSsim 模拟器的使用方法(见附录 B),然后了解 MIPSsim 的指令系统(见附录 C)。

1. 启动 MIPSsim (用鼠标双击 MIPSsim.exe)。

2. 点击“配置”→“流水方式”,使模拟器工作在非流水方式下。

3. 阅读附录,熟悉 MIPSsim 模拟器的操作和使用方法。

可以先载入一个样例程序(在本模拟器所在的文件夹下的“样例程序”文件夹中),然后分别以单步执行一条指令、执行多条指令、连续执行、设置断点等方式运行程序,观察程序的执行情况,观察 CPU 中寄存器和存储器的内容的变化。

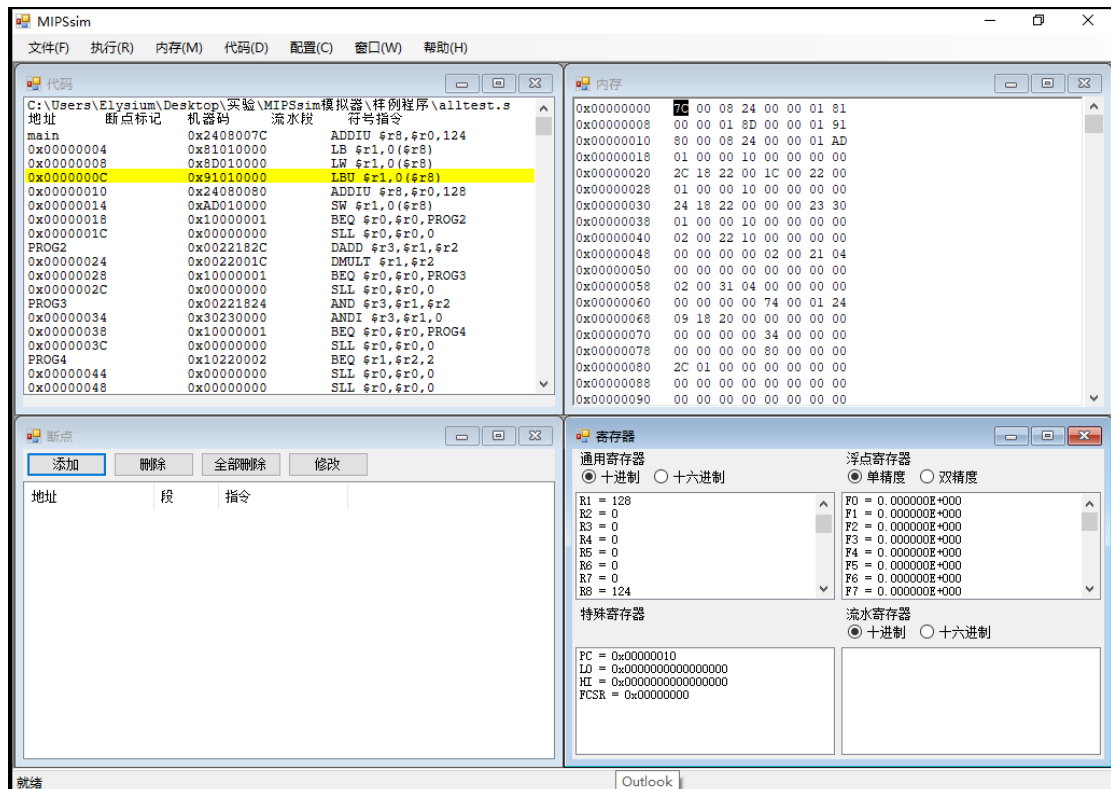
4. 选择“文件”→“载入程序”选项,加载样例程序 alltest.s,然后查看“代码”窗口,查

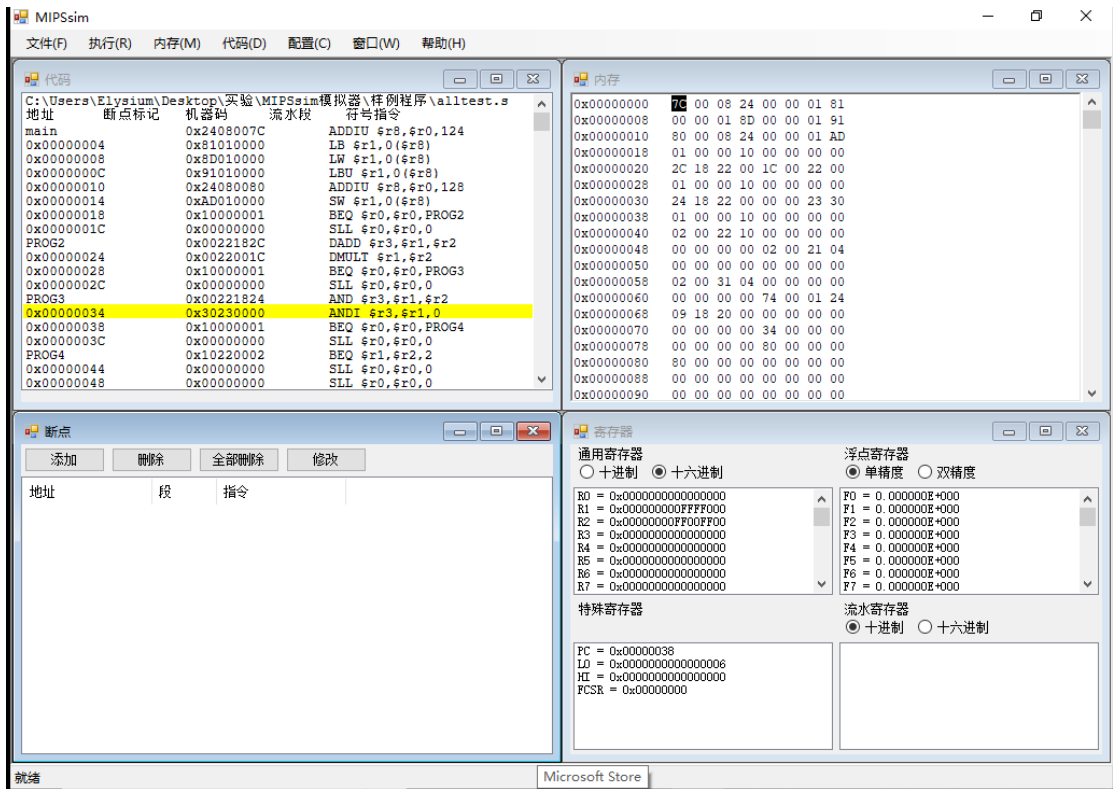
看程序所在的位置（起始地址为 0x00000000）。

5. 查看“寄存器”窗口 PC 寄存器的值：[PC] = 0x 0000 0000。
6. 执行 load 和 store 指令。步骤如下：
 - (1) 单步执行一条指令（F7）。
 - (2) 下一条指令地址为 0x 0000 0004，是一条有（有，无）符号载入字节（字节，半字，字）指令。
 - (3) 单步执行 1 条指令（F7）。
 - (4) 查看 R1 的值，[R1] = 0x FFFFFFFF80。
 - (5) 下一条指令地址为 0x 0000 0008，是一条有（有，无）符号载入字（字，半字，字）指令。
 - (6) 单步执行 1 条指令。
 - (7) 查看 R1 的值，[R1] = 0x 00000080。
 - (8) 下一条指令地址为 0x 0000 000C，是一条无（有，无）符号载入字节（字，半字，字）指令。
 - (9) 单步执行 1 条指令。
 - (10) 查看 R1 的值，[R1] = 0x 00000080。
 - (11) 单步执行 1 条指令。
 - (12) 下一条指令地址为 0x 0000 0014，是一条保存字（字，半字，字）指令。
 - (13) 单步执行 1 条指令。
 - (14) 查看内存 BUFFER 处字的值，值为 0x 80 0000 00。
7. 执行算术运算类指令。步骤如下：
 - (1) 双击“寄存器”窗口里的 R1，将其值修改为 2。
 - (2) 双击“寄存器”窗口里的 R2，将其值修改为 3。
 - (3) 单步执行 1 条指令。
 - (4) 下一条指令地址为 0x 0000 0020，是一条加法指令。
 - (5) 单步执行 1 条指令。
 - (6) 查看 R3 的值，[R3] = 0x 00000005。
 - (7) 下一条指令地址为 0x 0000 0024，是一条乘法指令。
 - (8) 单步执行 1 条指令。
 - (9) 查看 LO、HI 的值，[LO] = 0x 00000006，[HI] = 0x 00000000。
8. 执行逻辑运算类指令。步骤如下：
 - (1) 双击“寄存器”窗口里的 R1，将其值修改为 0xFFFF0000。
 - (2) 双击“寄存器”窗口里的 R2，将其值修改为 0xFF00FF00。
 - (3) 单步执行 1 条指令。
 - (4) 下一条指令地址为 0x 0000 0030，是一条逻辑与运算指令，第二个操作数寻址方式是寄存器直接寻址（寄存器直接寻址，立即数寻址）。
 - (5) 单步执行 1 条指令。
 - (6) 查看 R3 的值，[R3] = 0x FF00 0000。
 - (7) 下一条指令地址为：0x 0000 0034，是一条逻辑与指令，第二个操作数寻址方式是立即数寻址（寄存器直接寻址，立即数寻址）。
 - (8) 单步执行 1 条指令。
 - (9) 查看 R3 的值，[R3] = 0x 0000 0000。
9. 执行控制转移类指令。步骤如下：

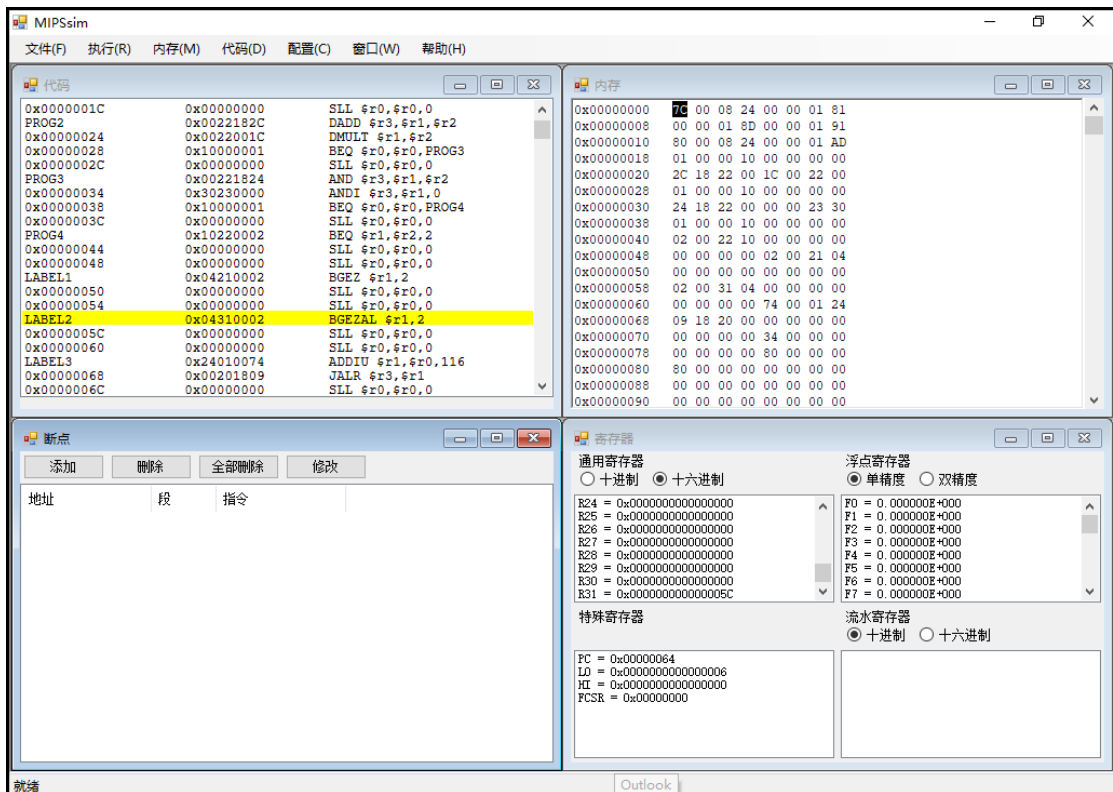
- (1) 双击“寄存器”窗口里的 R1, 将其值修改为 2。
- (2) 双击“寄存器”窗口里的 R2, 将其值修改为 2。
- (3) 单步执行 1 条指令。
- (4) 下一条指令地址为 0x 0000 0040, 是一条 BEQ 指令, 其测试条件是 [R1] = [R2], 目标地址为 0x 0000 004C。
- (5) 单步执行 1 条指令。
- (6) 查看 PC 的值, [PC]=0x 0000 004C, 表明分支 成功 (成功, 失败)。
- (7) 下一条指令地址是一条 BGEZ 指令, 其测试条件是 [R1] >= 0, 目标地址为 0x 0000 0058。
- (8) 单步执行 1 条指令。
- (9) 查看 PC 的值, [PC]=0x 0000 0058, 表明分支 成功 (成功, 失败)。
- (10) 下一条指令地址是一条 BGEZAL 指令, 其测试条件是 [R1] >= 0, 目标地址为 0x 0000 0064。
- (11) 单步执行 1 条指令。
- (12) 查看 PC 的值, [PC]=0x 0000 0064, 表明分支 成功 (成功, 失败);
查看 R31 的值, [R31]=0x 0000 005C。
- (13) 单步执行 1 条指令。
- (14) 查看 R1 的值, [R1]=0x 0000 0074。
- (15) 下一条指令地址为 0x 0000 0068, 是一条 JALR 指令, 保存目标地址的寄存器为 R1, 保存返回地址的目标寄存器为 R3。
- (16) 单步执行 1 条指令。
- (17) 查看 PC 和 R3 的值, [PC]=0x 0000 0074,
[R3]=0x 0000 006C。

三. 实验结果分析





执行 ANDI R1, R2, 0 指令, 观察到执行后 R3 被清零。该指令采用了立即数寻址。
操作数 0 直接被编码在指令中, 无需访问内存, 执行效率极高。在 MIPS 程序设计, 这是
寄存器初始化清零的常用手段。



以 `BGEZAL R1, 2` 为例。当 `[R1] ≥ 0` 时，观察到 PC 发生了非顺序跳转（跳过了 2 条指令地址）。R1 寄存器自动更新为 `BGEZAL` 下一条指令的地址。这体现了 MIPS 的链接机制。这种设计允许程序在执行完子程序后，能够精确返回断点继续执行。这是分支指令，也是实现函数调用的核心逻辑。

四. 结论

通过对寄存器状态和 PC 跳转的实时监控，验证了 MIPS 指令集在处理数据扩展、寻址及位级程序流控制上的精确性。